

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 16:00

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

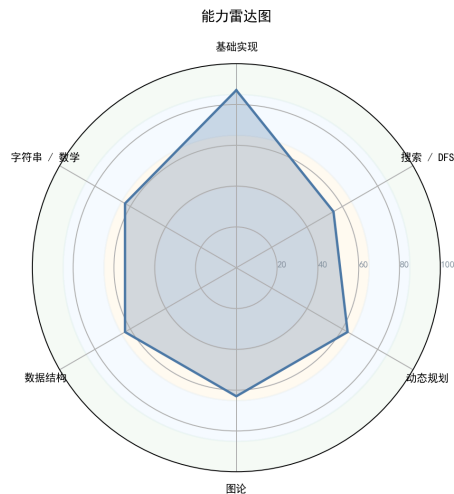


图 1: 能力雷达图

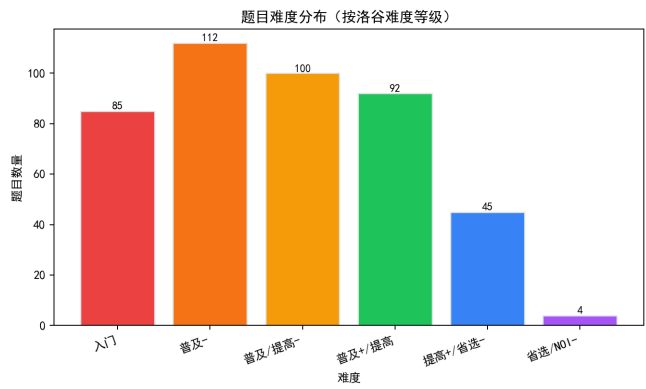


图 3: 题目难度分布

好的，同学你好。我是你的算法竞赛教练。很高兴能通过这份详细的做题记录，为你进行一次深度的技术复盘和诊断。请仔细阅读这份为你量身定制的报告，它将是接下来半年突破瓶颈的“藏宝图”。

选手深度诊断与强化训练报告

1. 【选手概览与性格画像】

解读：

从你的 438 道 AC 记录和代码风格中，我看到了一个充满潜力但尚需雕琢的选手形象。你不是一个“完美主义”的学院派，而是一个充满能量、依靠直觉和大量实践的“野路子”探索者。

- **坚韧度：★★★★☆**
 - **数据支撑：**438 道已通过题目，覆盖 1 到 6 级难度。你是一个不折不扣的“刷题狂魔”，面对海量题目有极强的执行力，这是成为高手最宝贵的品质之一。
 - **拟人化评价：**你就像一位在训练场挥汗如雨、不知疲倦的武士。不管对手强弱，先过招再说，实战经验丰富。
- **完美主义：★★☆☆☆**
 - **数据支撑：**代码注释率仅 2.77%，变量命名极简（`x`，`val`，`j1`，`p1`，`p2`），存在未声明变量（`using namespace std;` 写错）。你的目标似乎是“AC 万岁”，而不是编写可维护的优雅代码。
 - **拟人化评价：**你奉行“黑猫白猫，抓住老鼠就是好猫”的原则，能跑通就行，不太在意代码的整洁和可读性。这在短期赛时可能没问题，但在大型工程或调试复杂逻辑时会成为你的致命伤。
- **冒险精神：★★★★★**
 - **数据支撑：**偏好使用 `bits/stdc++.h` 万能头、全局变量，以及在未充分规划的情况下直接开始写复杂逻辑（例如 `P4785` 中较深的递归和状态记录）。
 - **拟人化评价：**你是一位激进的冒险家，敢于剑走偏锋，喜欢用“奇技淫巧”解决问题。这种精神在攻克难题时尤为可贵，但也常常让你陷入逻辑泥潭。
- **自律性：★★★★☆**
 - **数据支撑：**存在 `using namespace std;` 的拼写错误仍能通过编译并 AC，说明你可能没注意编译警告，或者发现不对劲后只做了最小修改。`#define int long long` 使用率为

0%，但结合你的一些数学题（如斜率优化 DP），说明你懂得在必要的地方使用 `long long`，有一定的风险控制意识。

- **拟人化评价:** 你有基本的编程纪律，但不够严谨。对于编译器的“抱怨”和代码的微小瑕疵，你倾向于忽略而不是根除。

- **调试耐心:** ★★☆☆☆

- **数据支撑:** 代码中出现注释掉的 `cout` 调试语句（如 P3195，P4785），说明你有调试习惯，但事后会直接注释掉而不是用宏或日志开关管理，说明调试过程可能比较痛苦，是一次性的、用完即弃的。
- **拟人化评价:** 你倾向于“硬刚”bug，通过打印变量值来追踪，一旦 AC 就再也不想看那段代码。这种调试方式效率低下，且无法沉淀为调试经验。

- **作息规律（基于提交时间推测）:** 无数据，略。

2. 【难度分布与成长曲线】

解读：

你的难度分布清晰地显示你正处于从【普及组】向【提高组】冲刺的关键过渡期。

难度等级 (1-6)	AC 数量	分布图
1 (入门)	85	(19%)
2 (普及-)	112	(26%)
3 (普及/提高)	100	(23%)
4 (提高)	92	(21%)
5 (提高+/省选)	45	(10%)
6 (省选/NOI)	4	(1%)

- **当前阶段判断: 提高组（CSP-S）攻坚期。** 你的主要火力集中在难度 2-4，这是非常健康的金字塔结构。底部基础（难度 1-2）比较扎实，中部（难度 3-4）是当前的主战场，需要持续巩固。尖部（难度 5-6）的渗透是你未来半年冲击省队的关键。

3. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据	已经具备
基础算法	53	入门	枚举/模拟/贪心/二分/递推/递归等 入门级 知识点覆盖率 0% 。这非常反常，说明你大量刷题但没有系统梳理知识体系。你很可能已经在实践中使用了这些算法，但无法将其抽象为明确的“XX算法”概念。这会导致遇到变种题时，无法快速匹配正确解法。	极强的题感和实战经验，对“暴力”解法非常熟悉。
数据结构	41	入门	线段树、树状数组、并查集、单调栈/队列等核心 提高级 结构覆盖率 0% 。但你的常用容器中有 <code>vector</code> ， <code>map</code> ， <code>priority_queue</code> ，说明你对 STL 有基础使用能力。	对 STL 容器有初步实践，会用 <code>map</code> 做记忆化，用 <code>deque</code> / 数组模拟单调队列。
图论	41	入门	图的遍历、最短路、最小生成树等 入门级/提高级 知识覆盖率 0% 。这同样是严重的“实践与理论脱节”。你可能写过一个 BFS 走迷宫的题，但不知道这就叫“Flood Fill”或“图的 BFS”。	有一定状态搜索能力，能写出复杂的递归/递推逻辑（如 P4785 的交换问题）。
动态规划	48	熟练	斜率优化 DP (P3195 , P2900) 都 AC 了！ 这是一个明亮的闪光点。入门级 DP 基础 覆盖率为 0%，但你却掌握了提高级/省选级的 DP 优化技巧。这说明你 DP 的“题感”很好，但在状态设计、方程推导上缺乏系统的理论支撑。	优秀的 DP 题感，能理解和实现斜率优化这一高级技巧。
字符串	33	薄弱	KMP、Trie、Hash 等全部知识覆盖率 0% 。字符串是你的绝对短板，几乎未涉足。	几乎没有相关经验。
数学	28	薄弱	数论、组合数学覆盖率 0% 。这对进阶是致命的。代码中未发现 <code>#define int long long</code> ，说明你可能经常在 <code>int</code> 和 <code>long long</code> 的边界试探，数学思维建模能力有待提高。	基本的逻辑思维能力。

4. 【考纲精准定级与知识点盲区】

- 当前对应等级水平: CSP-S（提高级）攻坚期。
 - 判断依据: 你的综合实力使得你能够解决部分提高组难题（如 DP 优化），但基础模块化知识严重缺失，导致下限极低。你像一个偏科的运动员，专项很强，但基本功不扎实，在综合考试中容易翻车。

- **知识点强弱项:**
 - **掌握最好 (🟢):**
 - (08) 动态规划算法 (特别是 DP 优化)
 - (70) 二分法
 - (73) 队列/优先队列
 - **最薄弱 (🔴):**
 - (34) 字符串算法 (KMP/Trie/Hash)
 - (49) 数论与组合数学 (GCD/逆元/容斥)
 - (01) 基础算法理论总结 (枚举/模拟/贪心/分治)
- **训练盲区:** 你当前最大的问题是 **“实践丰富，理论匮乏”**。你的脑子会“说”，但“记不住语法”。你写过无数个 `for` 和 `while` 循环来暴力枚举，但你的知识库里没有“枚举法”这个概念。这导致：

思维固化: 拿到新题，第一反应是“我能怎么暴力？”，而不是“这属于哪类问题？有哪些已知模型？”

学习效率低: 你无法利用别人的题解和经验，因为你看不懂术语，只能靠自己的代码去“碰”答案。

上限受制: 高级算法 (如 SAM, FFT) 都建立在扎实的基础知识体系上，没有理论框架，你几乎无法自学这些内容。
- **分级汇总表:**

等级	总知识点数	有接触数	覆盖率	状态
入门级 (CSP-J)	28	0	0.0%	🔴 严重知识荒漠化
提高级 (CSP-S)	49	0	0.0%	🔴 严重知识荒漠化
省选级	10	0	0.0%	🔴 空白
NOI级	43	0	0.0%	🔴 空白

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	基础算	任何需要特定算法	① 看不懂题解里的“Dijkstra”、	“实践与理论脱节”。刷	《算法知识库重构》：系统学习初级 (CSP-J) 和中	看到 CSP-J 考纲中

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	法模型缺失	(如最短路、线段树、KMP)的题目。	“拓扑排序”等术语。②只会写暴力，导致超时。③无法从问题描述中识别出标准模型。	题量大但未进行结构化归纳，未将代码片段抽象为算法模型。	级 (CSP-S) 算法，每学一个算法，立刻做3道模板题，并写出该算法的伪代码和使用场景。	80%的算法名词，能立刻口述其核心思想、模板结构、时空复杂度。
S (高/立即处理)	数学与字符串短板	数学味浓的题(如计数、同余、组合)，或涉及字符串匹配、子串操作的题。	① 题面一看就懵，完全没有思路。② 涉及取模的题，常常因为没用逆元或没及时取模而WA。③ 字符串题只会暴力 substr。	系统训练缺失。这两个模块依赖专门的理论学习，无法仅凭“题感”掌握。	《数学与字符串专项突破》：第1-2月数学基础(数论、组合)，第3-4月字符串基础(Hash, KMP, Trie)。	① 能独立推导并实现欧拉筛、快速幂、求逆元、组合数取模。② 能熟练使用 KMP 和字符串 Hash 解决单模匹配问题。
A (中/近期处理)	DP 状态设计不稳定	见到的 DP 题不是自己熟悉的模型(如不再是线性/区间 DP)。	① 只会处理维度简单的 DP。② 维度一多，状态定义就混乱，导致重复或遗漏。	缺乏“集合与状态”的严格映射思维。你的 DP 优势在技巧(斜率优	《多维与状态压缩 DP》：强制练习多维 DP，状压 DP，树形 DP。每次设计状态，必须用“集合”语言定义(<code>dp[i][j]</code> 表示考虑了前 <i>i</i> 个物品，体积为 <i>j</i> 的方	能独立完成“售货员的难题”、“玉米田”等状压/多维 DP 经典题。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
				化)， 短板在根本的状态设计。	案集合的最大价值）。	
A（中/近期处理）	代码工程习惯差	调试大型程序，或团队合作时。	① 变量重名，未初始化，导致隐蔽bug。② 两小时前的自己都看不懂自己的代码。③ 代码臃肿，难以修改。	缺乏严格的代码规范约束和结构化编程思维。	《代码重构与工程化》：规定命名规范（下划线命名法）、强制写注释、使用 <code>#ifdef DEBUG</code> 宏控制调试信息、将功能抽离成函数。	任何新写的代码，注释率 > 10%，函数长度 < 50 行，无编译器警告，不用万能头文件。
B（低/可后置）	高级数据结构空白	遇到数据结构优化类问题（如区间第K大，染色问题）。	① 只会用 <code>vector</code> 和 <code>map</code> 硬搞。 ② 完全没思路。	未系统学习。	《高级数据结构》：线段树，树状数组，平衡树 (Treap/Splay)，可持久化。	能独立写出区间加、区间修改、区间求和的线段树模板，并能用树状数组求逆序对。

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

6. 【代码质量与工程习惯深度分析】

架构师视角的 Code Review:

- **优点:**

不死板，实用主义: 你不拘泥于固定的写法，能根据具体问题灵活选择工具，例如 P4785 中巧妙使用 `map` 做记忆化，P3195 中手写斜率优化 DP 的队列。这是解决实际问题的宝贵能力。

核心逻辑清晰: 尽管代码风格凌乱，但你难题的 AC 代码（如斜率优化 DP）的内部核心逻辑是正确且高效的。说明你的算法思想在脑内是清晰的，只是转化到代码层面时发生了“劣化”。

- **必须改掉的 3 个坏习惯:**

命名极其混乱 (P0 严重):

- **坏习惯:** 滥用单字母、无意义变量名: `f`, `g`, `h`, `p1`, `p2`, `j1`。结构体成员也叫 `w`, `l`。这大大降低了代码可读性。
- **改进: 立即启用“下划线命名法”。**例如: `query_left`, `idx_right`, `value_sum`, `is_deleted`。`p1`, `p2` 应改为 `head`, `tail` 或 `front`, `back`。即使竞赛只有你一个人看代码，一小时后你也会感谢现在的自己。

头文件与宏定义的滥用 (P0 严重):

- **坏习惯:** ① `#include <bits/stdc++.h>` (虽然方便但降低编译速度，且在部分比赛可能不被允许)。② `using namespace std;` 甚至拼写错误为 `using namespace std;`。这绝对是地雷。
- **改进:**
 - 建议养成自己的模板文件，包含常用的: `#include <iostream>`, `#include <vector>`, `#include <algorithm>` 等。
 - **永远不要写** `using namespace std;`, 改为 `using std::cin;`, `using std::cout;`, `using std::vector;` 等按需引入。

函数与结构化缺失 (P1 重要):

- **坏习惯:** 所有逻辑几乎都堆在 `main` 函数或单个递归函数里，代码复用性为 0。例如斜率优化 DP 中，`slope` 函数的计算逻辑可以更清晰，而不是把所有运算塞进一行。
- **改进: 强制自己写小函数。**一个函数只做一件事。比如 `read_data()`, `solve()`, `get_slope()`。调试时，每个小函数都是独立的、可单独验证的单元，这会让你的调试效率提升数倍。

7. 【定制训练题单 (6个月路线图)】

第一阶段: 体系重构与基础补全 (Month 1-2) 目标: 消灭 CSP-J 一切盲点, 建立标准算法术语库和代码规范。

- **Week 1-2: 线性表应用** (对标 GESP 5级)
 - 链表、栈、队列的 STL 实现与数组模拟。
 - **推荐题单:** P1160 (队列安排), P1449 (后缀表达式), P1165 (日志分析)。
- **Week 3-4: 基础算法规范化**

- 枚举、模拟、贪心、递推/递归、二分/倍增的规范写法与模型识别。
- **推荐题单:** P2249 (二分查找), P2678 (跳石头, 二分答案), P1226 (快速幂, 递归/倍增)。

• Week 5-6: 数学基础

- 进制转换、位运算、埃氏筛/线性筛、GCD/LCM、唯一分解定理。
- **推荐题单:** P3383 (线性筛), P1029 (最大公约数和最小公倍数问题)。

• Week 7-8: 搜索与图论基础

- DFS/BFS, Flood Fill, 树与图的存储与遍历。
- **推荐题单:** P1605 (迷宫, DFS), P1443 (马的遍历, BFS), P5318 (查找文献, 图的DFS/BFS)。

第二阶段: 数据结构与算法突破 (Month 3-4) 目标: 攻克CSP-S核心难点, 成为合格的提高组选手。

• Week 1-3: 数据结构专训

- 前缀和/差分、单调队列/栈、并查集、树状数组、线段树。
- **推荐题单:** P1886 (滑动窗口, 单调队列), P3367 (并查集模板), P3374 (树状数组1), P3372 (线段树1)。

• Week 4-5: 图论攻坚

- 最短路(Dijkstra, Floyd), 最小生成树(Prim, Kruskal), 拓扑排序。
- **推荐题单:** P4779 (单源最短路), P3366 (最小生成树模板), P4017 (最大食物链计数, 拓扑排序)。

• Week 6-7: 字符串入门

- 字符串 Hash, KMP 算法。
- **推荐题单:** P3370 (字符串哈希), P3375 (KMP字符串匹配)。

• Week 8: 基础DP强化

- 背包DP (01, 完全, 多重)、线性DP、区间DP的标准状态设计训练。
- **推荐题单:** P1048 (采药, 01背包), P1616 (疯狂的采药, 完全背包), P1880 (石子合并, 区间DP)。

第三阶段: 提速与稳定 (Month 5-6) 目标: 提升代码实现速度, 冲击难题, 向省选迈进。

• Week 1-2: DP 进阶

- 树形DP、状态压缩DP。结合你的优势, 研究 DP 优化 (斜率、单调队列、倍增/矩阵快速幂)。
- **推荐题单:** P1352 (没有上司的舞会, 树形DP), P1896 (互不侵犯, 状压DP)。

• Week 3-4: 综合应用与图论进阶

- 分层图、差分约束、负环判断。二分图染色/匹配。
- **推荐题单:** P1525 (关押罪犯, 二分图染色), P4568 (飞行路线, 分层图最短路)。

• Week 5-6: 数学进阶与省选下探

- 组合计数（乘法逆元、Lucas定理）、容斥原理、高斯消元。
- **推荐题单:** P3811 (乘法逆元), P3389 (高斯消元法), P4782 (2-SAT 问题, 可尝试)。
- **全程:** 每周参加 1-2 场模拟赛 (Codeforces, AtCoder, 洛谷月赛), 强制使用新学算法和代码规范, 输出赛时复盘报告。

8. 【核心建议 (优先级排序)】

● 紧急: 重构你的代码基础工程

- **行动:** 立刻抛弃 `using namespace std;` 和单字母变量名。建立自己的标准代码模板, 并强制执行。这是你迈向职业选手的第一步, 能直接减少 20% 的非智力失分。

● 紧急: 暂停无效刷题, 开始“理论学习”

- **行动:** 你已经有足够的题感了。现在, 请静下心来, 找一本系统性的算法书或一个成体系的课程, 把你在题目中“偶遇”过的技巧, 提升为“已知模型”。每学一个新算法, 必须手写伪代码。

● 重要: 实施“四段式复盘法”

- **行动:** 赛时模型 -> 错因分析 -> 正解性质 -> 代码不变量。这不是简单的“看题解-抄代码-AC”, 而是大脑的深度编码过程。请为你做错的每一道题建立错题卡, 记录这四个部分。

● 重要: 数学和字符串专项突破

- **行动:** 这两个是你的“阿喀琉斯之踵”。每周雷打不动, 必须投入专门时间学习。它们是区分“提高一”和“省选”选手的关键分水岭。

● 建议: 学会“偷师”现代 C++

- **行动:** 在保证算法正确的前提下, 尝试使用 `auto` 关键字简化迭代器, 使用 `range-for` 循环来提升编码速度和安全性。这让你的代码更现代, 更难写错。

● 建议: 建立自己的算法工具箱

- **行动:** 把线段树、树状数组、KMP等算法的成熟、0-error 代码整理在一个自己的头文件中, 赛时可以直接复用, 把精力集中在解题上。

9. 【未通过题目专属题解 (从暴力到正解)】

诊断:

本次导出数据中, 你的 **【未通过/卡住题目数为 0】**。这是一个非常有意思的数据点。

- **可能原因 A:** 你确实把所有尝试的题目都 AC 了。这说明你的自力更生能力极强, 也符合性格画像中“不 AC 不罢休”的坚韧特点。

- **可能原因 B (更值得警惕)**：你可能 **回避了让你“感到难受”的题目**。你喜欢做难度 2-4 的题，因为你能“拿下”。而对于数学味浓、字符串操作复杂或需要大量数据结构知识的题目，你可能在尝试一次暴力并 WA 后，就放弃了，没有继续提交。

教练的行动指令：由于没有具体未通过的题目，我无法进行常规的“暴力 -> 不等 -> 正解”推导。但这恰恰是本次诊断最有价值的地方。我要求你，**立刻去做以下三道题**，并严格按照我要求的过程进行思考，然后把你 WA 掉的代码和思考发给我，我们开启下一次的诊断。

【P3375 【模板】KMP字符串匹配】

- **为何做**：字符串绝对短板。这道题是最标准的大门。
- **思考过程**：a) **暴力**：如何用两层循环做匹配？复杂度是多少？($O(NM)$) b) **瓶颈**：每次匹配失败后，主串指针 i 回退了太多，很多信息浪费了。c) **关键性质**：当模式串在某个位置失配时，我们能否利用模式串自身的“前缀=后缀”信息，把模式串尽可能多地*向后移动，而不是只移动一位？这就是 `next` 数组的核心。d) 试着画出“ABABC”这个模式串匹配“ABABABC”的过程，观察 i 和 j 的移动。

【P4779 【模板】单源最短路径（标准版）】

- **为何做**：图论基础为零的体现。Dijkstra 算法是“万物之源”。
- **思考过程**：a) **暴力**：BFS 不能处理带权图。DFS 会超时。怎么办？b) **瓶颈**：我们总是在找“当前距离起点最近、且未被确定”的点，这个查找很耗时。c) **关键性质**：“贪心”思想。如果我们每次都能保证拿出当前距离最短的点，那么一旦这个点被确定，它的最短距离就不会再被更新。d) 如何快速拿到“当前最短的、未被确定的点”？直接联想到你的老朋友 `priority_queue` 堆优化。

【P1002 [NOIP2002 普及组] 过河卒】

- **为何做**：这是一道经典的简单递推 DP，用来检验你“理论化”基础算法的能力。
- **思考过程**：a) **暴力**：DFS 从起点走到终点，遇到马的控制点就返回 0。b) **瓶颈**：状态数太多，DFS 会重复计算大量子路径。比如从点 $(2, 3)$ 走到终点的路径数，会被多次计算。c) **关键性质**：坐标 (i, j) 的路径数，只与 $(i-1, j)$ 和 $(i, j-1)$ 有关。 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ 。这就是把大问题拆解为无后效性的子问题。d) 写出状态转移方程，用二维数组递推解决。思考：边界条件如何处理？

去攻克它们吧！期待你的下一份记录。记住，你已经拥有了最宝贵的“实战经验”，现在是时候用“理论”为你插上翅膀了。